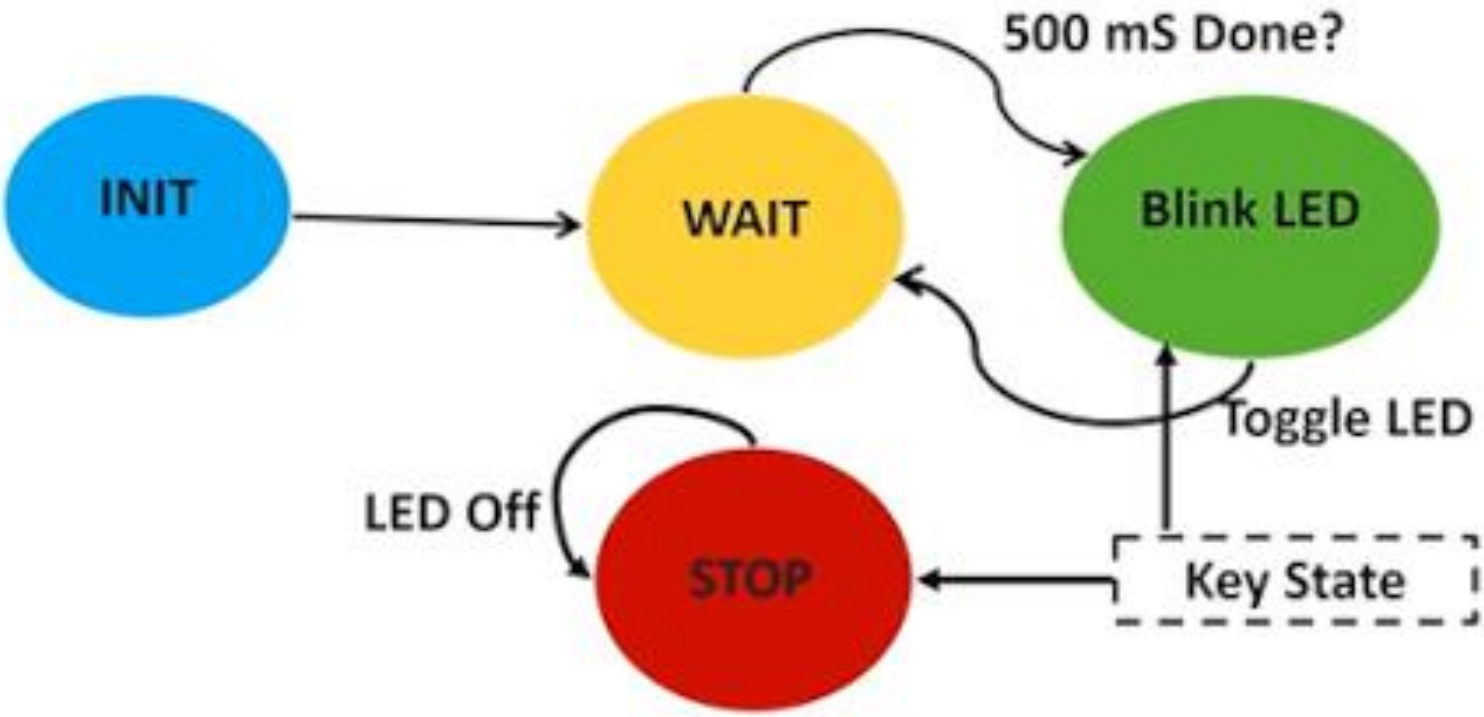
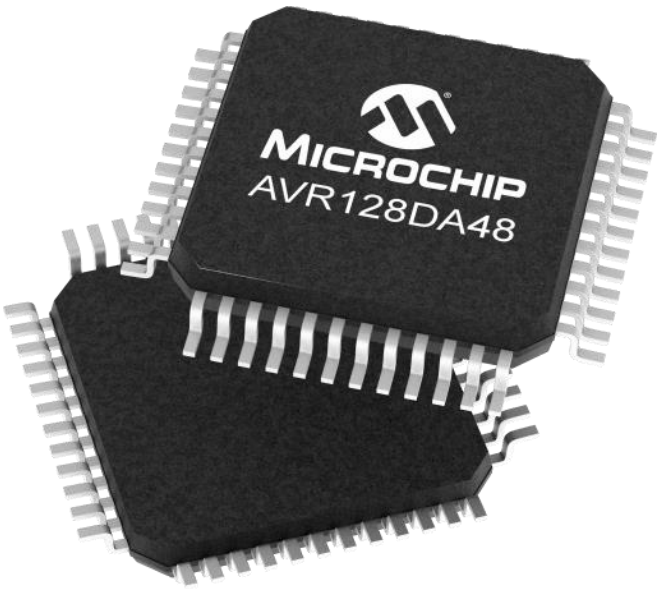
“Bare-Metal Embedded Systems with AVR128DA48 Course”

Day 27 - State Machines in Embedded Systems





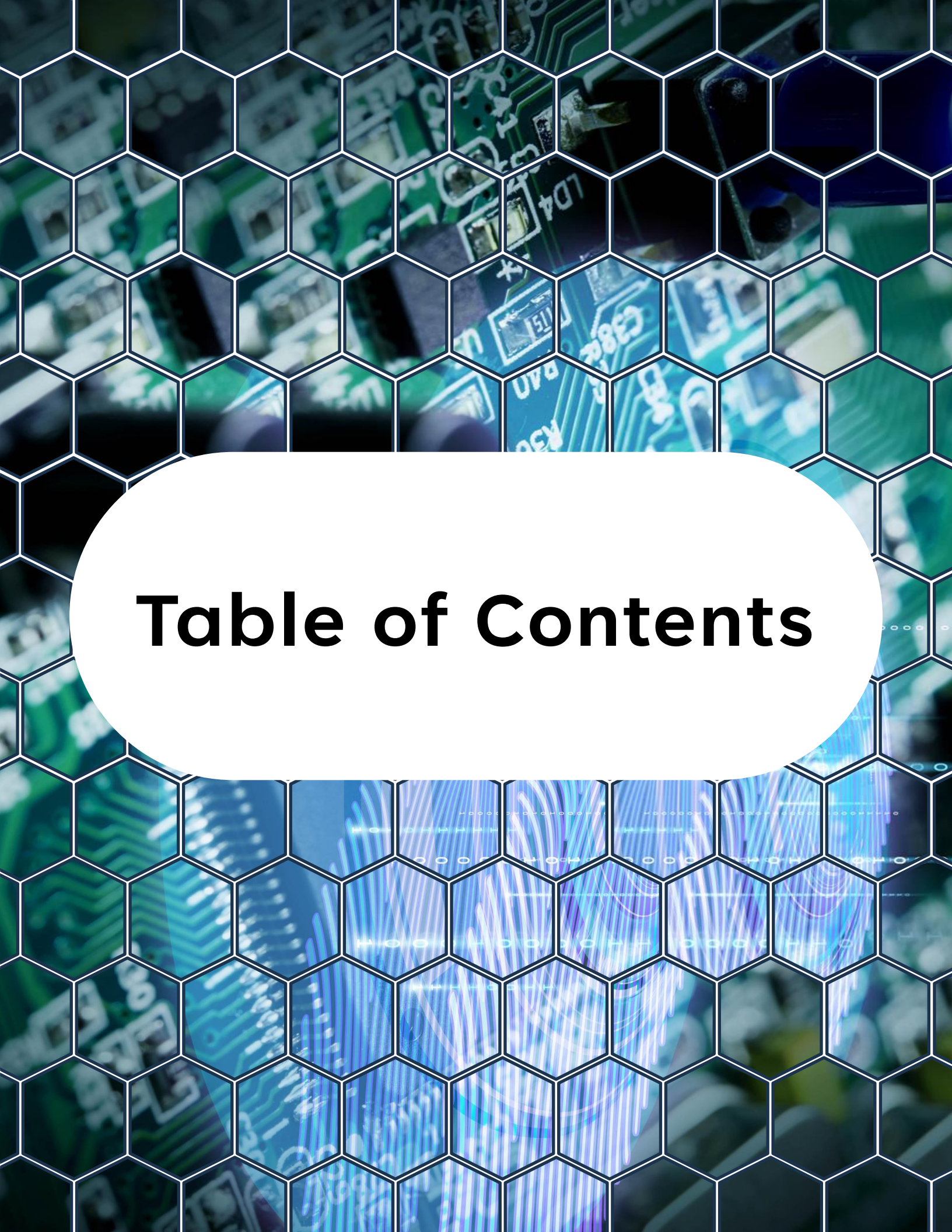
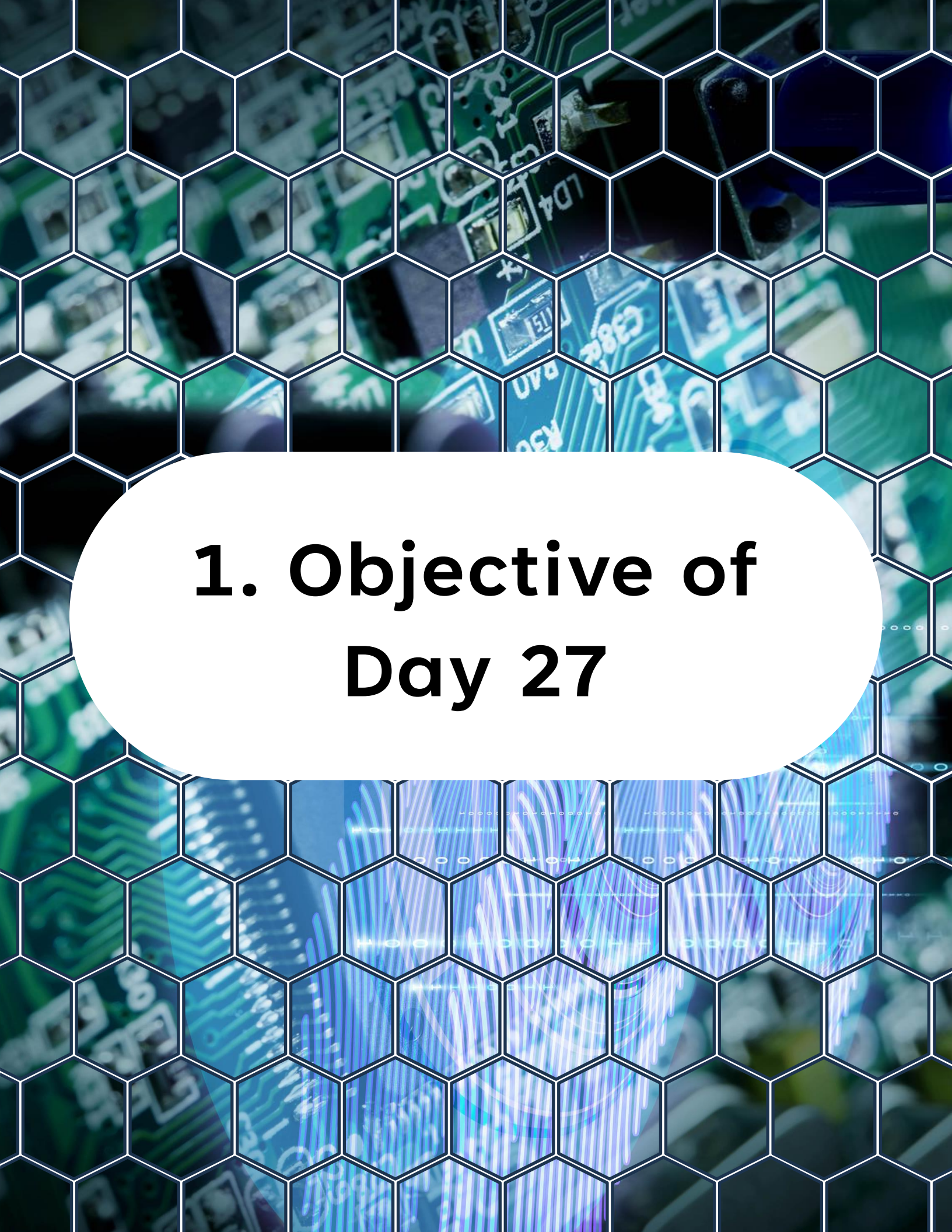


Table of Contents

Table of Contents



1. Objective of Day 27

1. Objective of Day 27

So far, your firmware has been mostly linear:

- Read input
- Process
- Act

That works for simple tasks.

But real embedded systems must handle:

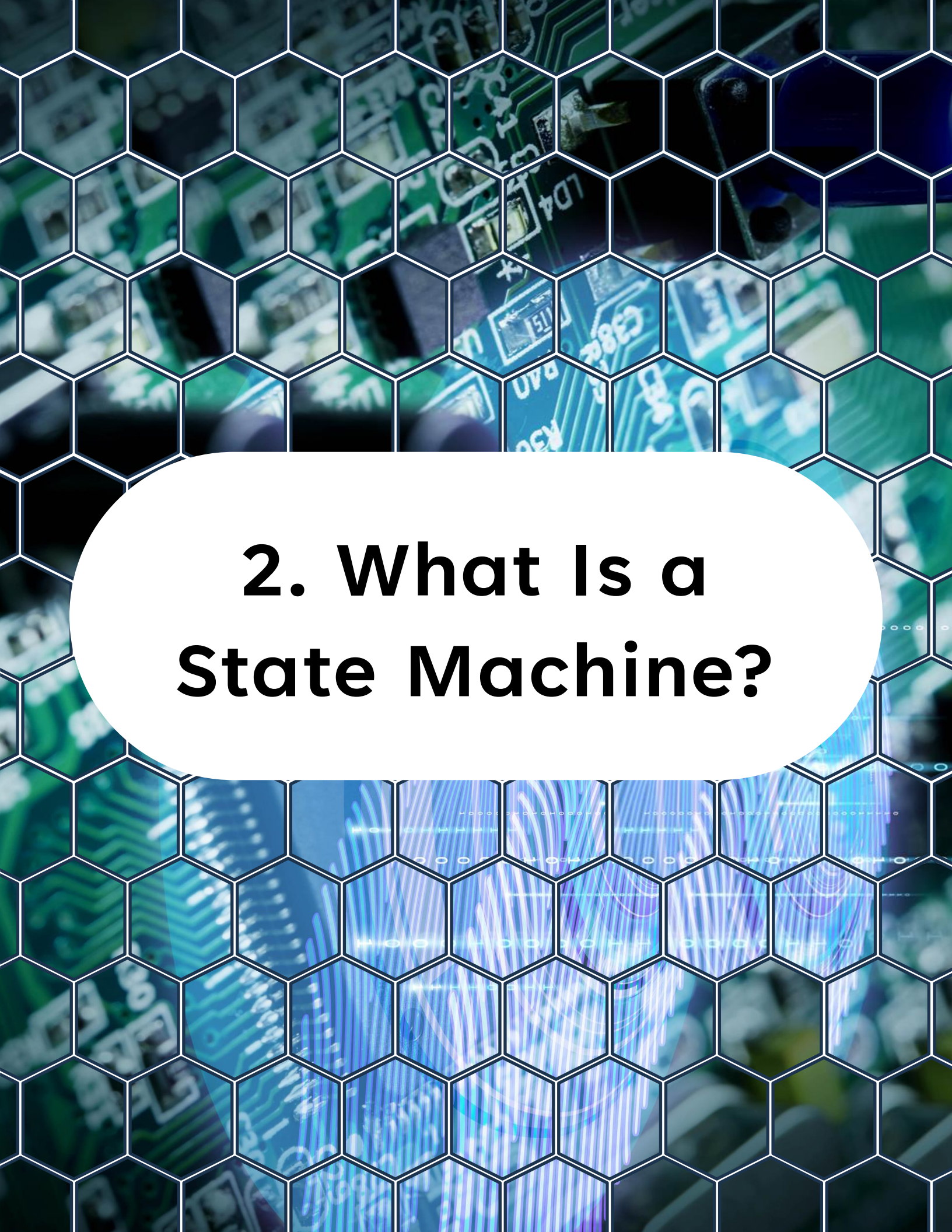
- Multiple conditions
- Asynchronous events
- Complex behavior
- Predictable control flow
- Today you will learn how to structure firmware using State Machines.

1. Objective of Day 27

By the end of today, you will understand:

- What a state machine is
- Why state machines are essential in embedded systems
- How to implement a finite state machine (FSM) in C
- How to handle events and transitions cleanly
- How to design scalable firmware behavior

This is one of the most important concepts in embedded software design.



2. What Is a State Machine?

2. What Is a State Machine?

A state machine is a model that describes behavior as:

- A set of states
- A set of transitions between states
- Actions that occur in each state

At any moment:

The system is in exactly one state.

2.1 Simple Example

Consider a system with:

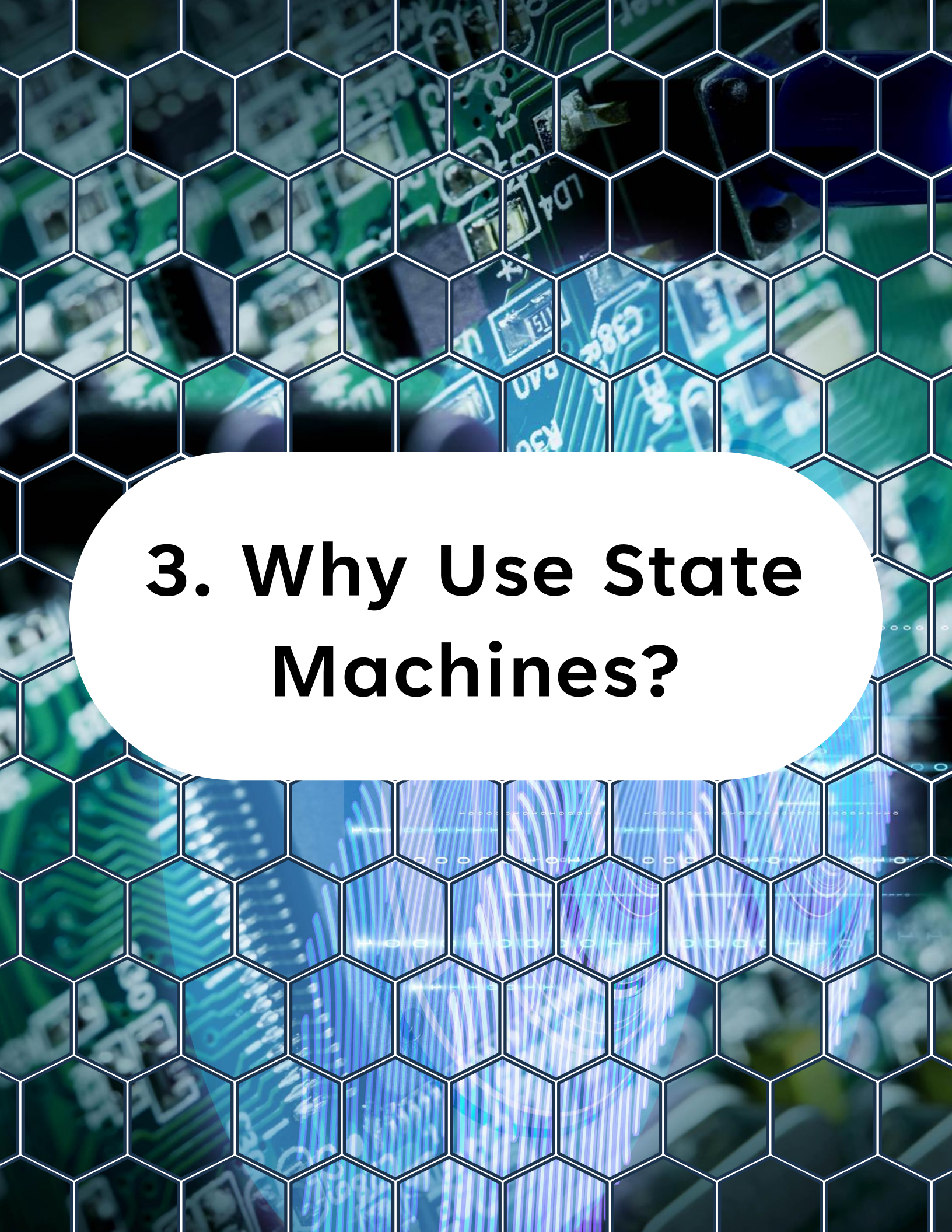
- IDLE
- RUNNING
- ERROR

2. What Is a State Machine?

Transitions:

- Button pressed -> IDLE -> RUNNING
- Fault detected -> RUNNING -> ERROR
- Reset -> ERROR -> IDLE

This is a finite state machine.



3. Why Use State Machines?

3. Why Use State Machines?

Without structure:

- Code becomes messy
- Conditions spread everywhere
- Hard to debug
- Hard to extend

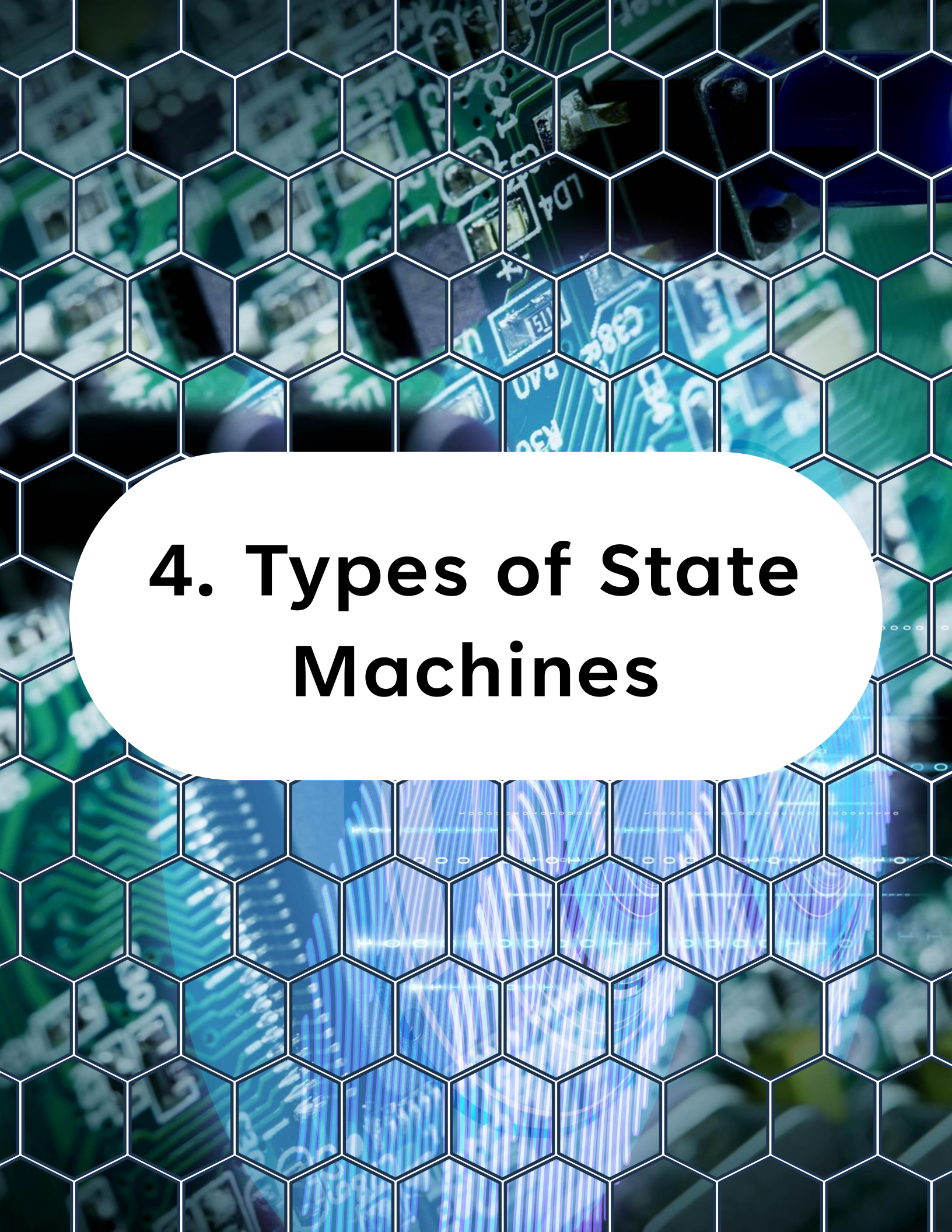
With state machines:

- Behavior is explicit
- Transitions are controlled
- Code is readable
- Easy to maintain

3. Why Use State Machines?

State machines are used in:

- Communication protocols
- Motor control
- User interfaces
- Power systems
- Embedded UI flows



4. Types of State Machines

4. Types of State Machines

4.1 Finite State Machine (FSM)

- Fixed number of states
- Simple transitions
- Most common

4.2 Hierarchical State Machine (HSM)

- States inside states
- Used for complex systems
- We focus on FSM today.

5. Basic FSM Implementation in C

5. Basic FSM Implementation in C

We define:

1. States (enum)
2. Current state variable
3. State handler (switch-case)

5.1 Define States

```c
/**
 * @brief System states.
 */
typedef enum
{
    STATE_IDLE,
    STATE_RUNNING,
    STATE_ERROR
} system_state_t;
```

5.2 State Variable

```c
static system_state_t current_state = STATE_IDLE;
```

5. Basic FSM Implementation in C

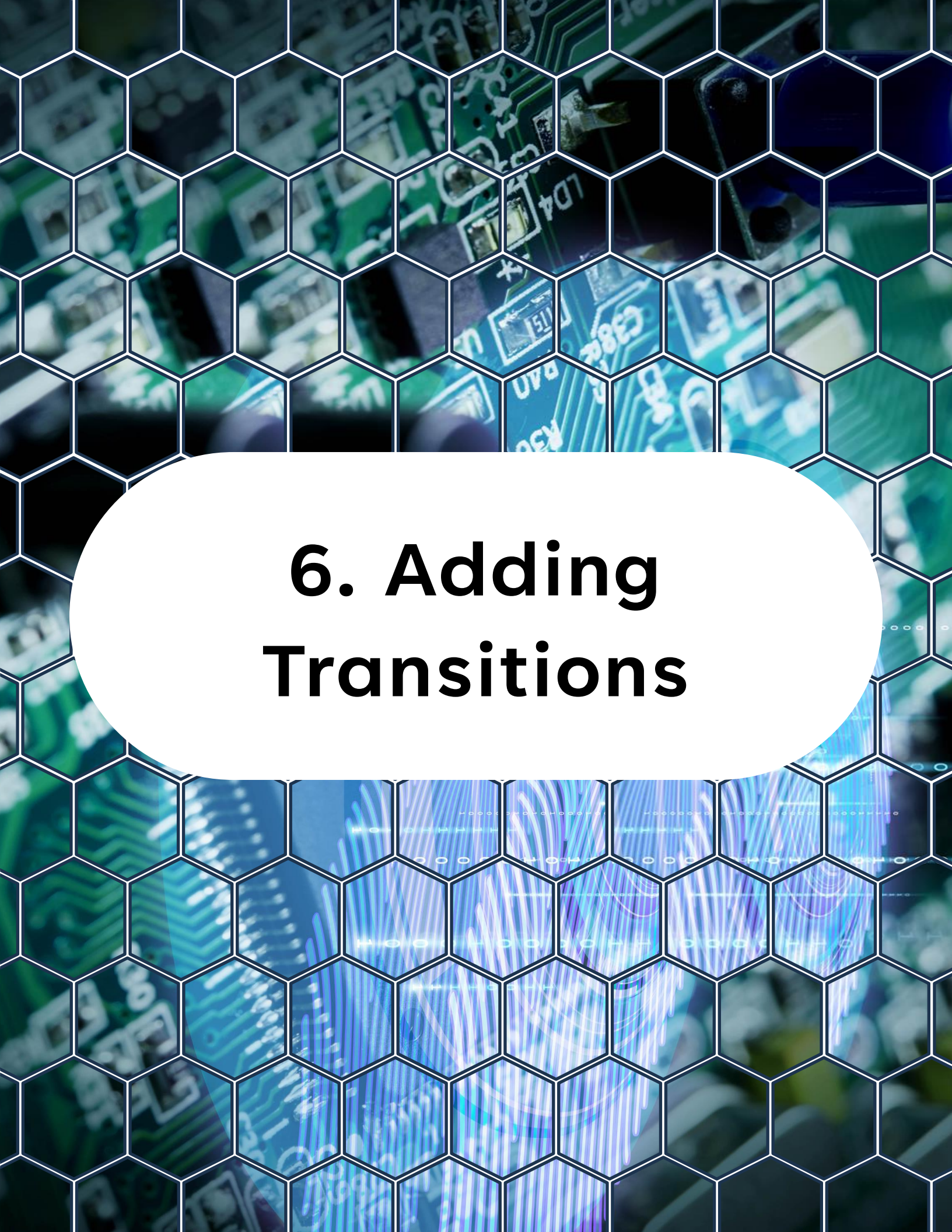
5.3 State Handler

```c
static void state_machine_run(void)
{
    switch (current_state)
    {
        case STATE_IDLE:
            /* Wait for start condition */
            break;

        case STATE_RUNNING:
            /* Perform main operation */
            break;

        case STATE_ERROR:
            /* Handle error */
            break;

        default:
            current_state = STATE_IDLE;
            break;
    }
}
```



6. Adding Transitions

6. Adding Transitions

Transitions are triggered by events:

- Button press
- Timer event
- ADC threshold
- Communication received

Example with Transitions

```c
static void state_machine_run(void)
{
    switch (current_state)
    {
        case STATE_IDLE:

            if (button_pressed())
            {
                current_state = STATE_RUNNING;
            }
            break;

        case STATE_RUNNING:

            if (fault_detected())
            {
                current_state = STATE_ERROR;
            }
            break;
```

6. Adding Transitions

```
21          case STATE_ERROR:
22
23              if (reset_condition())
24              {
25                  current_state = STATE_IDLE;
26              }
27              break;
28
29          default:
30              current_state = STATE_IDLE;
31              break;
32      }
33  }
```

7. State Entry and Exit Actions

7. State Entry and Exit Actions

Sometimes you need to execute code only once when entering a state.

Example with Entry Actions

```c
static system_state_t previous_state = -1;

static void state_machine_run(void)
{
    if (current_state != previous_state)
    {
        /* Entry actions */
        switch (current_state)
        {
            case STATE_IDLE:
                /* Initialize idle state */
                break;

            case STATE_RUNNING:
                /* Start operation */
                break;

            case STATE_ERROR:
                /* Log error */
                break;
        }

        previous_state = current_state;
    }

    /* State execution */
```

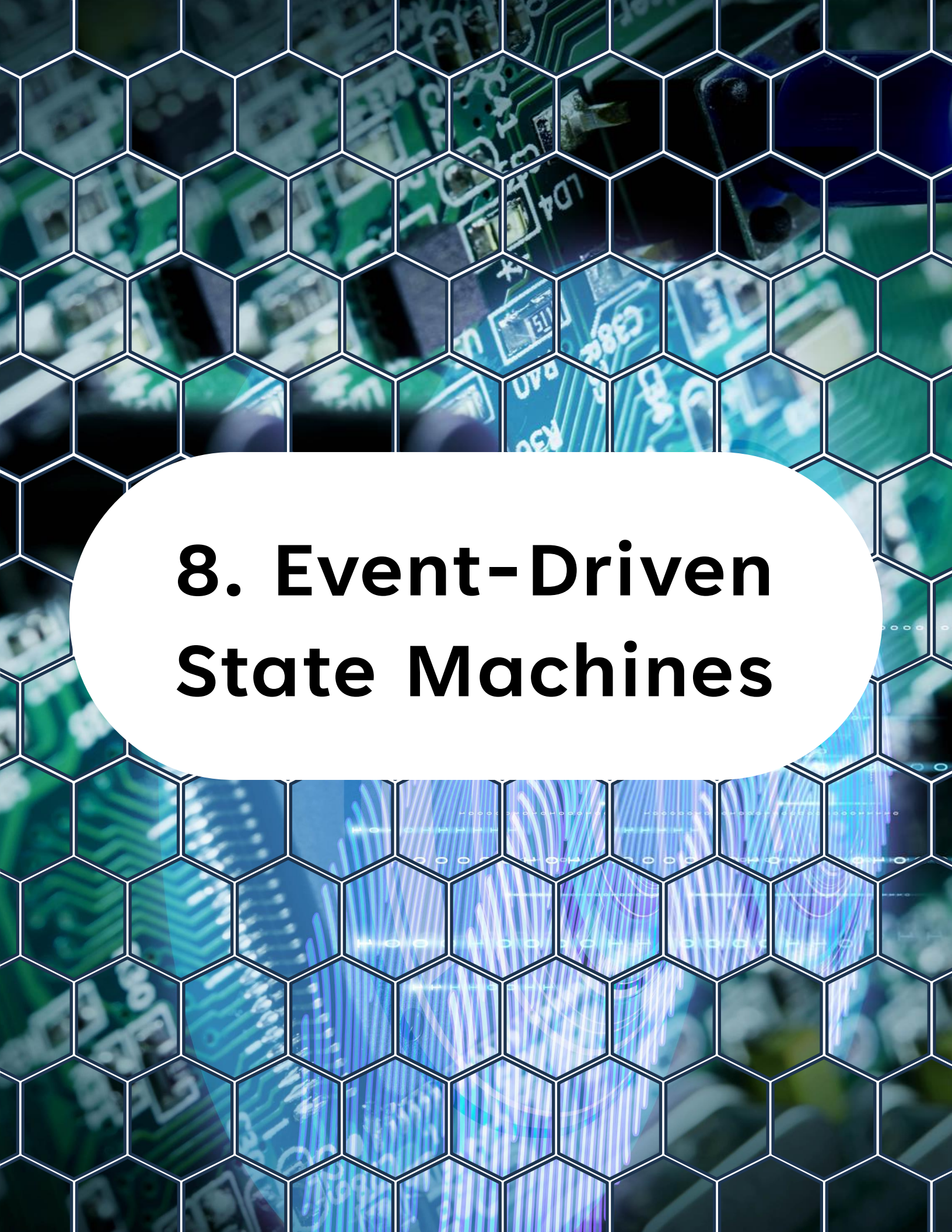
7. State Entry and Exit Actions

```
    /* State execution */
    switch (current_state)
    {
        case STATE_IDLE:
            if (button_pressed())
                current_state = STATE_RUNNING;
            break;

        case STATE_RUNNING:
            if (fault_detected())
                current_state = STATE_ERROR;
            break;

        case STATE_ERROR:
            if (reset_condition())
                current_state = STATE_IDLE;
            break;
    }
}
```

This is a clean and scalable pattern.



8. Event-Driven State Machines

8. Event-Driven State Machines

Instead of polling conditions, events can come from:

- Interrupts
- EVSYS
- Flags

Example:

```
volatile uint8_t event_flag = 0;

ISR(TCB0_INT_vect)
{
    event_flag = 1;
}
```

Then in main:

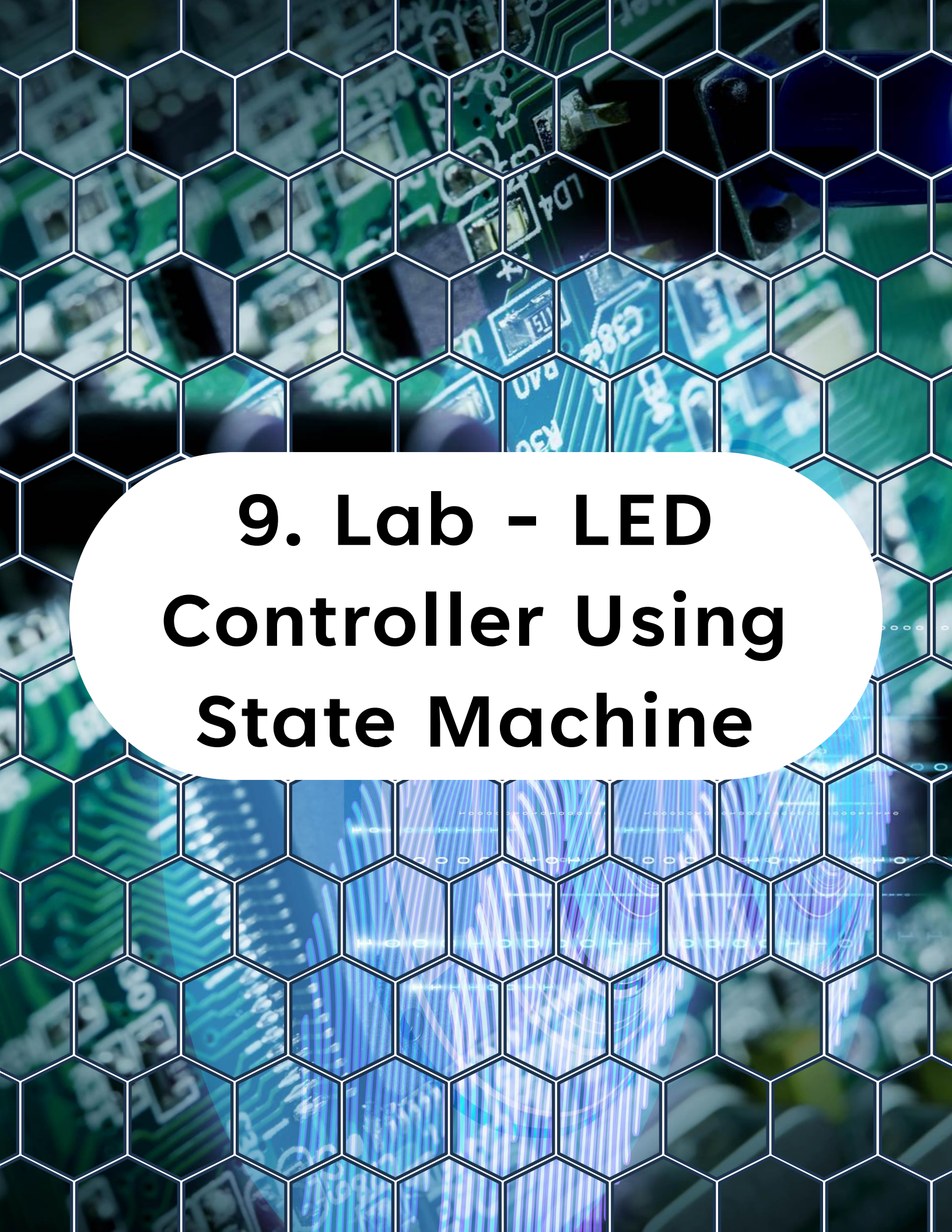
```
if (event_flag)
{
    event_flag = 0;
    /* Process event */
```

8. Event-Driven State Machines

Then in main:

```
if (event_flag)
{
    event_flag = 0;
    /* Process event */
}
```

State machines work very well with event-driven systems.



9. Lab – LED Controller Using State Machine

9. Lab - LED Controller Using State Machine

Goal

Create a system with:

- IDLE -> LED OFF
- RUNNING -> LED BLINK
- ERROR -> LED FAST BLINK

Transitions:

- Button press -> start
- ADC threshold -> error
- Reset button -> recover

9. Lab - LED Controller Using State Machine

Complete Example

```c
/**
 * @file main.c
 * @brief State machine example with LED control.
 */

#include <avr/io.h>
#include <stdint.h>

#define LED_PORT PORTA
#define LED_PIN  PIN7_bm

typedef enum
{
    STATE_IDLE,
    STATE_RUNNING,
    STATE_ERROR
} system_state_t;

static system_state_t current_state = STATE_IDLE;
static system_state_t previous_state = -1;

static void LED_init(void)
{
    LED_PORT.DIRSET = LED_PIN;
}

static void LED_toggle(void)
{
```

9. Lab - LED Controller Using State Machine

```
27 static void LED_toggle(void)
28 {
29     LED_PORT.OUTTGL = LED_PIN;
30 }
31
32 static void delay(uint32_t d)
33 {
34     for (volatile uint32_t i = 0; i < d; i++)
35     {
36     }
37 }
38
39 /* Mock conditions */
40 static uint8_t button_pressed(void)
41 {
42     return 0;
43 }
44
45 static uint8_t fault_detected(void)
46 {
47     return 0;
48 }
49
50 static uint8_t reset_condition(void)
51 {
52     return 0;
53 }
54
55 static void state_machine_run(void)
56 {
57     if (current_state != previous_state)
58     {
```

9. Lab - LED Controller Using State Machine

```
55 static void state_machine_run(void)
56 {
57     if (current_state != previous_state)
58     {
59         previous_state = current_state;
60     }
61
62     switch (current_state)
63     {
64         case STATE_IDLE:
65             if (button_pressed())
66                 current_state = STATE_RUNNING;
67             break;
68
69         case STATE_RUNNING:
70             LED_toggle();
71             delay(300000);
72
73             if (fault_detected())
74                 current_state = STATE_ERROR;
75             break;
76
77         case STATE_ERROR:
78             LED_toggle();
79             delay(100000);
80
81             if (reset_condition())
82                 current_state = STATE_IDLE;
83             break;
84
85         default:
86             current_state = STATE_IDLE;
```

9. Lab - LED Controller Using State Machine

```
85          default:
86              current_state = STATE_IDLE;
87              break;
88      }
89  }
90
91  int main(void)
92  {
93      LED_init();
94
95      while (1)
96      {
97          state_machine_run();
98      }
99  }
```

What You Should Observe

- LED behavior changes depending on state
- Transitions are controlled and predictable
- Logic is easy to follow

You can expand this system easily.

10. Best Practices

10. Best Practices

- Keep states simple
- Keep transitions explicit
- Avoid deeply nested conditions
- Separate state logic from hardware drivers
- Use enums for clarity

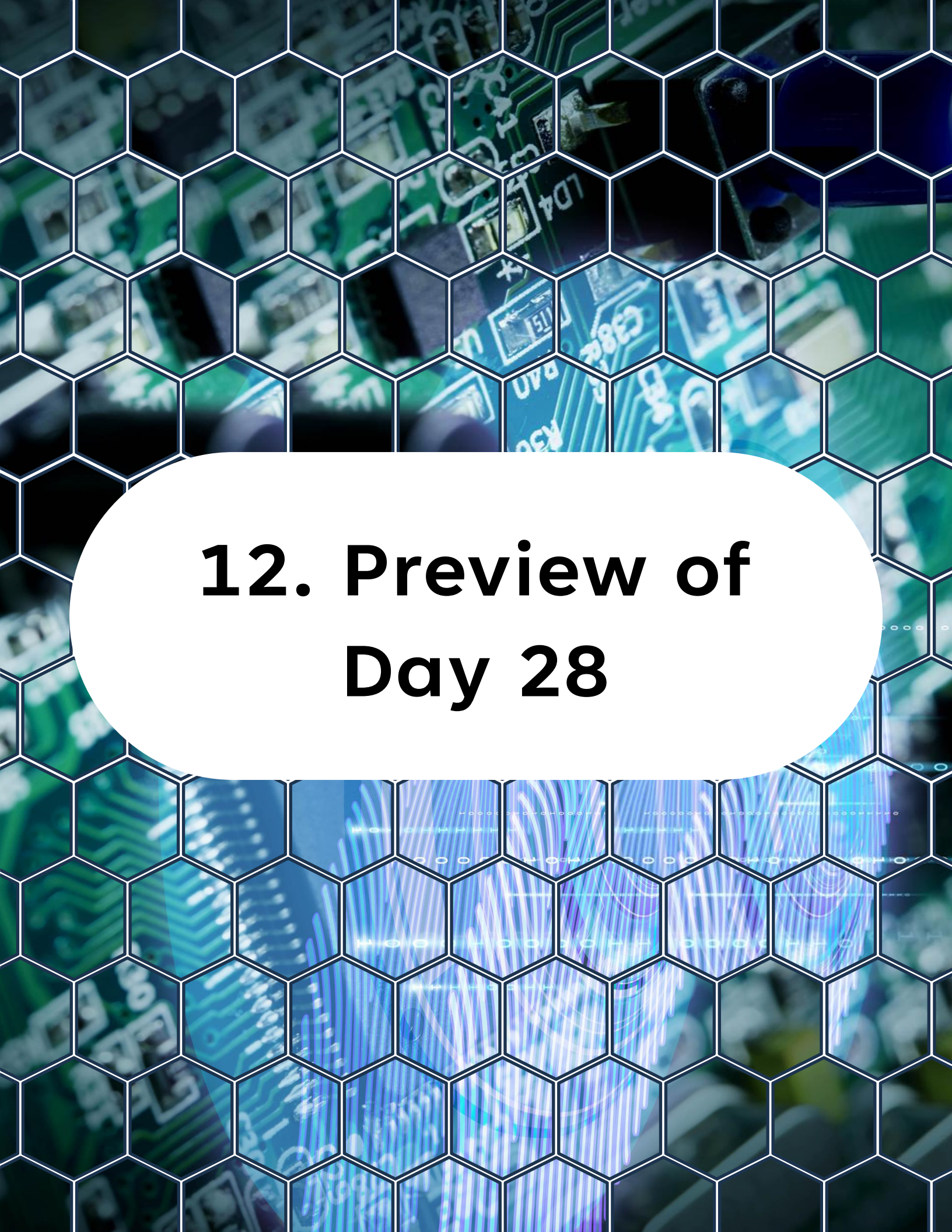
11. What You Should Understand Now

11. What You Should Understand Now

You now know:

- What a state machine is
- How to implement FSM in C
- How to handle transitions
- How to structure embedded logic
- How to design scalable firmware behavior

This is one of the most important tools in embedded software engineering.



12. Preview of Day 28

Preview of Day 28

Next:

Queues and Buffers in Embedded Systems

You will learn how to:

- Decouple modules
- Handle data streams
- Build robust communication pipelines

This is where systems become loosely coupled and scalable.

You can find all source code, examples, and updates for this course in the GitHub repository, make sure to visit it, explore the files, and clone it so you can follow along hands-on.

https://github.com/god233012yamil/Bare-Metal-Embedded-Systems-with-AVR128DA48-Course